Muhammad Haris Ahsan harismirza3456@gmail.com

ICSC Qualification Round

Solutions:

Problem A: A History of Computing

1. **(1936): Publication of Alan Turing's Seminal Paper / The Universal Turing Machine**

Description: Alan Turing published "On Computable Numbers, with an Application to the Entscheidungsproblem", introducing the concept of the Universal Turing Machine (UTM). This provided the mathematical framework and theoretical foundation for all modern programmable computers.

2. **(1947): Invention of the Transistor at Bell Labs**

Description: John Bardeen, Walter Brattain, and William Shockley invented the point-contact transistor at Bell Telephone Laboratories. This solid-state electronic component replaced fragile, power-hungry vacuum tubes and served as the fundamental building block of modern digital electronics.

3. **(1958): Invention of the Integrated Circuit (IC)**

Description: Jack Kilby at Texas Instruments (and independently Robert Noyce at Fairchild Semiconductor shortly after) invented the Integrated Circuit. By placing multiple transistors, resistors, and capacitors onto a single piece of semiconductor material, it paved the way for microchips.

4. **(1969): Creation of ARPANET / The Intel 4004 Development Begins**

Description: The Advanced Research Projects Agency Network (ARPANET) established its first node-to-node communication link, creating the foundational architecture of the modern Internet. (Alternatively: Intel began development on the 4004 microprocessor, which would launch in 1971).

5. **(1981): Launch of the IBM PC (Model 5150)**

Description: IBM introduced the IBM Personal Computer (Model 5150), running MS-DOS. This standardized personal computing architecture globally, driving mass adoption of PCs across businesses and homes.

6. **(1991): Public Availability of the World Wide Web (WWW)**

Description: Tim Berners-Lee released the World Wide Web project to the public on the internet. The introduction of URLs, HTTP, and HTML transformed the internet from an academic network into a globally accessible information platform.

7. **(2000): The Y2K Bug Transition / Launch of the USB Flash Drive**

Description: The global technology sector successfully navigated the millennial date transition (Y2K bug) without widespread infrastructure failures. The year 2000 also marked the commercial introduction of the USB flash drive, revolutionizing portable data storage.

8. **(2007): Introduction of the iPhone / Rebirth of Modern Smart Devices**

Description: Apple launched the original iPhone, combining a multi-touch smartphone, an internet communicator, and a media player. This event catalyzed the modern mobile computing era and shifted consumer technology toward app-driven ecosystems.

9. **(2012): The AlexNet Breakthrough / The Deep Learning Boom**

Description: Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton published AlexNet, a convolutional neural network that won the ImageNet challenge by an unprecedented margin. This event marked the beginning of the modern deep learning explosion and the AI revolution.

10. **(2022): Launch of ChatGPT and the Generative AI Explosion**

Description: OpenAI released ChatGPT to the public, setting off a massive global paradigm shift toward Generative Artificial Intelligence and Large Language Models (LLMs) integrated into software engineering and consumer productivity.

—---

Problem B: Pixel Art.

Python code implementing the generate_shape function to handle both the checkerboard and diamond patterns.

```
def generate_shape(n: int, shape: str) -> list[list[int]]:

    """
    Generates specific geometric patterns on an N x N grid.

    Parameters:
    n (int): The grid size (5 <= n <= 51, always odd for the diamond).
    shape (str): Either "checkerboard" or "diamond".

    Returns:
    list[list[int]]: A 2D list representing the grid filled with 0s and 1s.
    """

    grid = [[0 for _ in range(n)] for _ in range(n)]

    if shape == "checkerboard":
        for r in range(n):
            for c in range(n):
                if (r + c) % 2 != 0:
                    grid[r][c] = 1

    elif shape == "diamond":
        center = n // 2
        for r in range(n):
            for c in range(n):
```

```
                if abs(r - center) + abs(c - center) <= center:
                    grid[r][c] = 1

    return grid

def print_grid(grid: list[list[int]]):
    for row in grid:
        print(" ".join(map(str, row)))
```

Explanation for Solution

Checkerboard Logic:
A chessboard pattern naturally satisfies the mathematical condition where a cell at coordinate (r, c) alternates state based on the parity of its coordinates. Because the problem dictates that the top-left cell (0, 0) must be empty (0), any cell where r + c yields an odd number is designated as filled (1).

Diamond Logic:
Since N is strictly odd, the grid possesses a perfect central coordinate at ([N/2], [N/2]). A diamond shape is geometrically equivalent to a bounding region defined by the Manhattan distance. A grid point is filled if its total horizontal and vertical distance from the center is less than or equal to the radius [N/2].

—

Problem C: Moore's Law

(a) 1971 Prediction for the Year 2026

We use the classic Moore's Law formula:
`T(t) = T_o x 2^{t/2}`

Reference year (t=0): 1971 (Intel 4004)
Initial transistor count (T_o): 2,300
Target year: 2026
Elapsed time (t): 2026 - 1971 = 55 years

Substituting the values into the formula:
`T(55) = 2,300 x 2^{55/2} = 2,300 x 2^{27.5}`

Calculating the exponential factor:
`2^{27.5} = 189,812,534.2`

Multiplying by the initial count:
$T(55) \times 2,300 \times 189,812,534.2 = 436,568,828,660$

Predicted 2026 Transistor Count:
= 436.57 billion transistors.

(b) Comparison with an Actual 2026 Processor (Apple M5)

- **Actual Transistor Count:** Modern cutting-edge consumer microchips targeting 2025/2026 production cycles (such as scaled-up multi-die architectures or high-density mobile SoCs like the Apple M5 series) typically range from roughly 100 billion to over 400 billion transistors depending on the specific tier (e.g., standard vs. Ultra configurations).
- **Analysis:** Moore's Law holds up remarkably well over this 55-year span. The prediction of 436.6 billion sits right on the upper tier of what modern extreme-ultraviolet (EUV) lithography processes achieve today, demonstrating that the exponential scaling trend has remained highly accurate.

(c) Calculating the Best-Fit Doubling Period

To find the exact doubling period d that maps 2,300 transistors in 1971 to a representative modern count of, say, 150 billion (1.5 x 10^{11}) transistors over 55 years:
$1.5 \times 10^{11} = 2,300 \times 2^{55/d}$

Divide both sides by 2,300:

$65,217,391.3 = 2^{55/d}$

Take the base-2 logarithm ($\log_2$) of both sides:

$\log_2(65,217,391.3) = 55/d$
$25.959 = 55/d$

Solve for d:
$d = 55/25.959 = 2.12$ years

Best-Fit Doubling Period:
2.12 years (or roughly 25.4 months). This shows that the real-world growth rate was only slightly slower than the theoretical 2-year (24-month) doubling cycle.

—---

Muhammad Haris Ahsan harismirza3456@gmail.com

Problem D: The Simultaneous Strike

(a) What is wrong with the pseudocode?

The bug lies in processing events purely sequentially on a line-by-line basis without considering frame boundaries.Premature Break Condition:

1. The break check occurs at the start of processing each individual event rather than at the end of a frame.
2. Sequential vs. Simultaneous Processing: Because the events are processed one by one, if Player 1's attack event is processed first on frame 512, it reduces Player 2's HP to -5. On the very next iteration (which is Player 2's attack on the exact same frame 512), the condition if player1_hp <= 0 OR player2_hp <= 0 evaluates to true, and the loop terminates immediately.
3. The Consequence: Player 2's attack is completely ignored, denying them a legitimate double-KO and erroneously awarding the victory to Player 1 based solely on how the stable sort ordered their events.

(b) Corrected Python Implementation

Here is the robust, bug-free implementation of processGame. It groups or processes all actions occurring on the same frame completely before performing any health status or KO validation checks.

```
def processGame(events: list[tuple[int, int, int]], H: int) -> list[int]:
    """
    Replays match events in frame order, ensuring simultaneous strikes
    on the same frame are fully applied before checking for a KO.

    Parameters:
    events (list of tuples): (player, frame, attack_value)
    H (int): Starting health points for both players.

    Returns:
    list[int]: [player1_hp, player2_hp] clamped to a minimum of 0.
    """
    player1_hp = H
    player2_hp = H

    sorted_events = sorted(events, key=lambda x: x[1])

    i = 0
    n = len(sorted_events)

    while i < n:
        if player1_hp <= 0 or player2_hp <= 0:
            break

        current_frame = sorted_events[i][1]

        p1_damage_received = 0
        p2_damage_received = 0
```

```
        while i < n and sorted_events[i][1] == current_frame:
            player, frame, attack_value = sorted_events[i]
            if player == 1:
                p2_damage_received += attack_value
            elif player == 2:
                p1_damage_received += attack_value
            i += 1

        player1_hp -= p1_damage_received
        player2_hp -= p2_damage_received

    return [max(0, player1_hp), max(0, player2_hp)]
```

—

Problem E: PageRank in the Age of AI?

(a) Impact of AI-Generated Content SEO Networks on PageRank
The proliferation of AI-generated content farms engineered specifically to interlink with one another introduces a significant challenge called a sybil attack or an automated link farm.

- **Artificial Inflation**: PageRank mathematically distributes authority across directed links. When automated networks spin up thousands of synthetically generated web pages that point to a target page, they artificially inflate that target page's incoming link count and traffic metrics.
- **Dilution of Quality**: Because the PageRank algorithm assumes a link is an endorsement of quality by a human creator, large-scale, low-cost AI networks break this core assumption. They dilute the authority of genuine, high-quality human-curated sites by burying them under immense volumes of synthetic PageRank loops.

(b) Mathematical Mitigation Strategies
To defend the PageRank algorithm against AI link spam, structural modifications can be introduced:

- **TrustRank (Biased Random Surfer):** Instead of calculating PageRank uniformly across the web, seed the algorithm with a hand-verified directory of highly trusted, human-vetted authority sites (e.g., major universities, official government portals). The damping factor d is adjusted so that the random surfer acts as a Biased Random Walk, where the probability of jumping to a random page is strictly limited to this trusted seed set. The authority degrades exponentially the further a page is structurally removed from a trusted seed site, effectively containing AI link farms in a low-score bubble.
- **Anti-Trust Rank:** A complementary mathematical layer that seeds the algorithm with known AI spam nodes. Authority is propagated backward through incoming links to penalize pages that actively accept trust or distribute links to and from known automated networks.

~ End